

Almacenamiento de Datos Distribuidos - Guía

Contexto y Propósito General

Este documento aborda la evolución y los desafíos del almacenamiento de datos, desde sistemas tradicionales hasta soluciones distribuidas para Big Data. Se centra en cómo manejar volúmenes masivos de información (como los 90 PB anuales del LHC) mediante sistemas de ficheros distribuidos, RAID, formatos de archivo optimizados y bases de datos NoSQL. El objetivo es facilitar el estudio de tecnologías clave para el almacenamiento escalable y eficiente en entornos de datos masivos.

1. Sistemas de Ficheros

1.1 Concepto y Función

- **Definición:** Infraestructura que organiza y almacena datos en unidades de almacenamiento (discos duros, SSDs, etc.).
- **Unidad básica:** Fichero (archivo).
- **Estructura:** Árbol de directorios para organizar archivos en carpetas según su uso o significado.
- **Ejemplo práctico:** Permite guardar una factura y una carta en ubicaciones separadas, facilitando su recuperación.

1.2 Tipos Comunes

- **FAT32:** Usado en dispositivos portátiles.
- **NTFS:** Predeterminado en Windows.
- **EXT4:** Común en Linux.

1.3 Formatos de Archivo

- **Texto delimitado (CSV, TSV):** Ideal para datos estructurados legibles. `csv`
`FirstName,LastName,Email Joe,Jones,joe@litware.com`
 - **JSON:** Flexible para datos estructurados/semiestructurados con esquema jerárquico.
 - **XML:** Menos usado hoy, reemplazado por JSON.
 - **BLOB:** Para datos binarios (imágenes, audio, video).
 - **Formatos optimizados:**
 - **Avro:** Basado en filas, con compresión eficiente.
 - **ORC:** Basado en columnas, optimizado para Apache Hive.
 - **Parquet:** Basado en columnas, eficiente para datos anidados.
-

2. RAID (Redundant Array of Independent Disks)

2.1 Concepto

- Combina múltiples discos para mejorar capacidad, rendimiento o redundancia.
- Funciona por hardware (controladoras) o software (más lento pero económico).

2.2 Niveles Principales

- **RAID 0 (Striping):**
 - Ejemplo: 2 discos de 1TB → 1 unidad de 2TB.
 - Sin redundancia, mayor velocidad.
- **RAID 1 (Mirroring):**
 - Ejemplo: 2 discos de 1TB → 1 unidad de 1TB replicada.
 - Tolerante a fallos, velocidad de lectura mejorada.
- **RAID 5 (Striping con paridad distribuida):**
 - Ejemplo: 3 discos de 1TB → 1 unidad de 2TB con redundancia.
 - Tolerante a fallos de un disco, rendimiento mejorado.

3. Sistemas de Ficheros Distribuidos

3.1 Concepto

- Almacena archivos distribuidos en un clúster, permitiendo acceso unificado.
- Divide archivos en **bloques grandes** (ej. 128MB) distribuidos en nodos.
- **Ejemplo clave:** HDFS (Hadoop Distributed File System).

3.2 Características

- **Escalabilidad:** Archivos más grandes que el almacenamiento de cualquier nodo.
- **Redundancia:** Replicación automática de bloques en múltiples nodos (índice configurable, $N \geq 2$).
- **Rendimiento:** Lectura/escritura paralela, evitando cuellos de botella.
- **Limitación:** No ideal para muchos archivos pequeños; se recomienda consolidarlos en ETL.

3.3 Ejemplos en la Nube

- **Amazon S3**
- **Google Cloud Storage**
- **Azure Blob Storage**

4. Aplicación en Big Data: Caso LHC

- **Problema:** 90 PB de datos anuales generados por experimentos.
 - **Solución:** Uso de sistemas distribuidos como HDFS para almacenar y procesar datos masivos, combinado con formatos optimizados (Parquet, Avro) para eficiencia.
-

Puntos Clave para Estudio

1. **Sistemas de ficheros** organizan datos; los distribuidos escalan para Big Data.
 2. **RAID** mejora capacidad/seguridad pero no reemplaza la distribución en clústeres.
 3. **Formatos como Parquet/ORC** optimizan almacenamiento y consultas.
 4. **HDFS y soluciones cloud** son esenciales para datos a escala petabyte.
 5. La **replicación distribuida** elimina la necesidad de RAID a nivel local.
-

Resumen ejecutivo (≈100 palabras):

Esta guía cubre tecnologías de almacenamiento desde sistemas de ficheros tradicionales hasta soluciones distribuidas para Big Data. Explica RAID para redundancia local, formatos de archivo (JSON, Parquet) según el tipo de dato, y sistemas como HDFS que distribuyen bloques en clústeres para manejar petabytes (ej. LHC). Destaca la importancia de la replicación automática, el acceso paralelo y la consolidación de archivos pequeños. El objetivo es proporcionar bases para diseñar almacenamiento escalable y eficiente en entornos de datos masivos.

Resumen Detallado: Sistemas de Almacenamiento Distribuido para Big Data

1. Sistemas de Archivos Distribuidos

1.1. Concepto y Redundancia

Los sistemas de archivos distribuidos permiten almacenar grandes volúmenes de datos en múltiples nodos de un clúster. La **redundancia** se consigue replicando el contenido de cada archivo en más de un nodo (no en todos), garantizando disponibilidad ante fallos.

Ejemplo de redundancia: Un archivo de 1 GB puede dividirse en bloques replicados en 3 nodos diferentes. Si un nodo falla, los datos siguen accesibles desde las réplicas.

Sistemas mencionados: * **Google File System (GFS):** Precursor y base teórica. * **Amazon S3:** Servicio de almacenamiento de objetos en la nube. * **HDFS:** Sistema de archivos de Hadoop, detallado a continuación.

1.2. Almacenamiento Distribuido en Memoria (RAM)

Consiste en mantener los datos en la memoria RAM de los nodos del clúster para eliminar la latencia de E/S de los discos.

Ventajas: * **Velocidad extrema:** Lectura de datos ~100 veces más rápida que desde disco (ej: 0.2 ms vs 20 ms para 1 MB). * **Procesamiento en tiempo real:** Ideal para analítica compleja y flujos de datos rápidos (característica de velocidad del Big Data).

Desventajas y Consideraciones: * **Falta de durabilidad intrínseca:** Los datos en RAM se pierden si se apaga la máquina. **No se consigue durabilidad a menos que los datos se persistan en otro soporte.** * **Costo elevado:** Ampliar memoria requiere añadir nodos completos (escalado horizontal), lo que es costoso. * **Casos de uso adecuados:** Análisis exploratorio/iterativo, algoritmos de grafos, aplicaciones de baja latencia con transacciones ACID. * **Casos no adecuados:** Procesamiento por lotes (batch), presupuestos limitados, necesidad de persistencia inmediata de los datos de origen.

1.3. Hadoop Distributed File System (HDFS)

1.3.1. Características y Funcionamiento

HDFS es el sistema de archivos distribuido de código abierto de Apache Hadoop, diseñado para almacenar grandes volúmenes de datos en clústeres de hardware económico.

Principales Características: * **Schema-on-read:** No impone una estructura de datos al escribir, lo que permite gran flexibilidad. Se interpreta el esquema al leerlos. * **Bloques grandes:** Por defecto de **128 MB**. Esto minimiza los accesos (búsquedas) al disco, optimizando la **velocidad de transferencia** (throughput) en lugar de la latencia. Es ideal para procesamiento por lotes. * **Tolerancia a fallos:** Lograda mediante la replicación de bloques en varios nodos. * **Rack Awareness:** Conoce la topología de red (switches) del clúster para optimizar la localidad de los

datos y minimizar el tráfico entre racks. * **Hardware:** Está diseñado para funcionar en **máquinas comerciales estándar (commodity hardware)**, no requiere hardware de alto rendimiento o especializado.

Reflexión (Tamaño máximo de archivo): Con bloques de 128 MB y identificadores de 64 bits (2^{64} bloques únicos), el tamaño máximo teórico de un archivo en HDFS es astronómico: $128 \text{ MB} * 2^{64}$.

1.3.2. Acceso por Línea de Comandos

Se interactúa con HDFS principalmente mediante el comando `hadoop fs`, con una sintaxis similar a los comandos Unix/Linux.

Comandos básicos: * `hadoop fs -ls /` : Listar el directorio raíz. * `hadoop fs -mkdir /datos` : Crear un directorio. * `hadoop fs -copyFromLocal local.txt /datos/` : Copiar un archivo local a HDFS. * `hadoop fs -copyToLocal /datos/archivo.txt ./` : Copiar de HDFS al sistema local. * `hadoop fs -cat /datos/archivo.txt` : Mostrar el contenido de un archivo. * `hadoop fs -rm /datos/archivo.txt` : Eliminar un archivo.

Nota: También se puede usar `hdfs dfs` con un comportamiento ligeramente distinto.

1.3.3. Acceso desde Python

Se puede acceder mediante librerías específicas, como **Snakebite**, que implementa el protocolo Hadoop RPC para comunicarse directamente con el Namenode.

Ejemplo con Snakebite:

```
from snakebite.client import Client
client = Client('localhost', 9000)

# Listar un directorio
for x in client.ls(['/']):
    print(x)

# Leer un archivo de texto
for line in client.text(['/input/datos.txt']):
    print(line)
```

2. Bases de Datos NoSQL (Introducción)

El texto introduce el concepto de las bases de datos NoSQL como una solución al problema de la rigidez del modelo relacional. En un caso práctico, se muestra la dificultad de modelar en una sola base de datos relacional entidades como `Persona`, `Actor`, `Cantante` y `Piloto` que comparten atributos pero tienen propiedades específicas. Esto lleva a la creación de múltiples tablas interrelacionadas, un esquema complejo que las bases de datos NoSQL (documentales, grafos, clave-valor, etc.) buscan simplificar al permitir estructuras de datos más flexibles y escalables horizontalmente.

Propósito General: Esta unidad proporciona una base sobre las principales estrategias de almacenamiento distribuido en entornos Big Data, contrastando sistemas de archivos optimizados para capacidad y costo (HDFS) con almacenamiento en memoria para velocidad, e introduciendo la necesidad de bases de datos NoSQL para manejar datos variados y no estructurados de manera eficiente.

Bases de Datos NoSQL - Guía Completa

Resumen General

Este documento explora las bases de datos NoSQL como alternativa a las bases de datos relacionales tradicionales. Se aborda el problema del diseño rígido de esquemas (schema-on-write) en bases relacionales y se presenta la filosofía schema-on-read de NoSQL. Se detallan los cuatro tipos principales de bases NoSQL (documentales, clave-valor, columnares y de grafos), y se explican conceptos fundamentales de bases distribuidas como sharding y replicación. El objetivo es proporcionar una comprensión práctica de cuándo y por qué usar estas tecnologías, con ejemplos concretos y consideraciones sobre escalabilidad y disponibilidad.

1. El Problema con las Bases de Datos Relacionales

1.1. Schema-on-Write vs. Schema-on-Read

- **Bases Relacionales (Schema-on-Write):** Requieren definir la estructura de las tablas (atributos, tipos de datos) ANTES de escribir los datos. Esto implica un diseño previo que puede ser problemático si no se conoce completamente el dominio del problema.
- **Bases NoSQL (Schema-on-Read):** Permiten escribir documentos sin un esquema predefinido. El esquema se interpreta en el momento de la lectura. Esto ofrece flexibilidad para manejar datos semiestructurados o no estructurados.

1.2. Limitación Práctica

- **Ejemplo:** Modelar personas con múltiples profesiones (ej: cantante y actor) y sus relaciones de amistad (un grafo) es complejo en un modelo tabular relacional, pero natural en una base de datos de grafos.
-

2. ¿Qué son las Bases de Datos NoSQL?

2.1. Definición

- Significa "No Solo SQL" o "No Relacionales".
- Conjunto de tecnologías que escapan al modelo relacional tradicional.
- **NO** implica que no usen SQL (algunas sí usan variantes), sino que no están basadas en el modelo relacional.

2.2. Tipos Principales

a) Bases de Datos de Documentos

- Almacenan datos como documentos (normalmente JSON, BSON, XML).
- Cada documento puede tener una estructura diferente.

- **Ejemplo:** Un documento `Cliente` con campos `nombre`, `email`, y `direcciones` (un array). `json { "id": 1, "nombre": "Ana García", "email": "ana@ejemplo.com", "profesiones": ["Ingeniera", "Bloguera"] }`
- **Base de datos ejemplo:** MongoDB, Couchbase.

b) Bases de Datos Clave-Valor

- Estructura simple: una clave única asociada a un valor (que puede ser cualquier cosa: texto, número, objeto, etc.).
- Muy rápidas para operaciones de lectura/escritura simples.
- **Ejemplo:** Una caché de sesiones de usuario. Clave: `"session_abc123"` Valor: `"{userId: 456, lastLogin: '2023-10-27'}"`
- **Base de datos ejemplo:** Redis, DynamoDB.

c) Bases de Datos Columnares (Familia de Columnas)

- Almacenan datos en tablas, filas y columnas, pero las columnas se agrupan lógicamente en "familias".
- Optimizadas para consultas agregadas sobre grandes volúmenes de datos (análisis).
- **Ejemplo:** Una tabla para pedidos donde una familia de columnas almacena `info_cliente` (nombre, dirección) y otra almacena `info_producto` (nombre, precio).
- **Base de datos ejemplo:** Cassandra, HBase.

d) Bases de Datos de Grafos

- Modelan datos como **nodos** (entidades) y **aristas** (relaciones).
 - Ideales para datos altamente interconectados (redes sociales, recomendaciones, fraud detection).
 - **Ejemplo:** Modelar una red social. Los nodos son `Personas` y las aristas son `AMIGO_DE` o `TRABAJA_CON`.
 - **Base de datos ejemplo:** Neo4j, Amazon Neptune.
-

3. Conceptos Clave para Bases Distribuidas (NoSQL)

3.1. Sharding (Particionamiento)

- **Qué es:** Dividir un conjunto de datos grande en fragmentos más pequeños (**shards**) que se distribuyen en diferentes nodos/servidores de un clúster.
- **Objetivos:**
 1. **Manejar datasets más grandes** que la capacidad de un solo servidor.
 2. **Escalabilidad horizontal:** Distribuir la carga de trabajo.
 3. **Tolerancia parcial a fallos:** Si un nodo cae, solo se pierde acceso a una parte de los datos.
- **Desafío:** Diseñar una **estrategia de sharding** (ej: por rango de ID, por hash) que intente que los datos que se consultan juntos residan en el mismo shard para minimizar operaciones entre nodos.

3.2. Replicación

- **Qué es:** Mantener copias idénticas de los datos (**réplicas**) en múltiples nodos.
- **Objetivos:**
 1. **Alta disponibilidad:** Los datos son accesibles desde varios puntos.
 2. **Tolerancia a fallos:** Si un nodo se cae, sus réplicas en otros nodos siguen sirviendo los datos.
 3. **Escalabilidad en lecturas:** Las consultas de lectura se pueden repartir entre réplicas.

Estrategias de Replicación:

- **Maestro-Esclavo (Master-Slave):**
 - **Escrituras:** Solo en el nodo **maestro**. Luego se propagan a los **esclavos**.
 - **Lecturas:** Se pueden hacer en cualquier nodo (usualmente en esclavos para aliviar al maestro).
 - **Punto único de fallo:** El nodo maestro.
 - **Ideal para:** Cargas de trabajo intensivas en lecturas.

- **Problema:** Posible inconsistencia temporal si se lee de un esclavo antes de que se actualice.
- **Par-a-Par (Peer-to-Peer):**
 - **Sin maestro:** Todos los nodos son iguales (**peers**).
 - **Escrituras/Lecturas:** Se pueden realizar en cualquier nodo. Las escrituras se replican después a los demás.
 - **Sin punto único de fallo.**
 - **Problema:** Posibles conflictos de escritura si dos nodos modifican el mismo dato simultáneamente.
 - **Gestión Pesimista:** Usa bloqueos para evitar conflictos (reduce disponibilidad).
 - **Gestión Optimista:** Permite conflictos temporales, los resuelve después (puede perderse una escritura si se basa en un estado obsoleto).

3.3. Sharding con Replicación (Combinación Ideal)

- Se combinan ambas técnicas para obtener sus ventajas.
- Cada **shard** tiene sus propias **réplicas** distribuidas en el clúster.
- **Ejemplo:** Un nodo puede ser el **maestro del Shard A** y a la vez un **esclavo del Shard B**.
- Proporciona **escalabilidad, disponibilidad y tolerancia a fallos**.

4. Conclusión y Aplicación Práctica

- **Elige NoSQL cuando:** Necesites flexibilidad de esquema, escalabilidad horizontal masiva, manejar datos no relacionales (documentos, grafos) o velocidades extremas de lectura/escritura simple.
- **Mantén SQL/Relacional cuando:** Tus datos sean altamente estructurados, necesites transacciones complejas (ACID) y consistencia fuerte, y tus relaciones sean bien definidas y estables.
- **Conceptos como Sharding y Replicación** son clave para entender el rendimiento y resiliencia de los sistemas de datos modernos, aunque no son exclusivos de NoSQL.

Ejemplo de Decisión: Para una red social donde las relaciones entre usuarios son centrales, una base de **grafos** (Neo4j) sería más natural y eficiente para consultas como "amigos de amigos". Para el catálogo de productos de un e-commerce con atributos variables, una base de **documentos** (MongoDB) ofrece más flexibilidad que una tabla SQL rigidamente definida.

Resumen Detallado: Bases de Datos NoSQL y MongoDB

1. Introducción a las Bases de Datos NoSQL

Las bases de datos NoSQL son sistemas de almacenamiento que no siguen el modelo relacional tradicional. Surgieron para abordar necesidades específicas de almacenamiento y gestión de información que las bases de datos relacionales no cubrían eficientemente. Su principal característica es la **flexibilidad** en el manejo de datos, especialmente útil en escenarios con datos heterogéneos o de gran volumen.

Ejemplo práctico: En una aplicación de redes sociales, donde los datos de cada usuario pueden variar significativamente (algunos tienen muchos amigos, otros muchas publicaciones, otros información laboral detallada), una base de datos NoSQL permite almacenar esta información sin una estructura rígida predefinida.

2. Las Cuatro Familias Principales de Bases de Datos NoSQL

2.1. Bases de Datos Documentales

- **Concepto:** Almacenan pares **clave-documento**, donde el documento suele estar en formato JSON y no requiere un esquema predefinido (schema-on-read).

- **Estructura:** En lugar de tablas, usan **colecciones** que agrupan documentos similares.
- **Ejemplo de documento:**

```
json { "docId": "893852383254", "Tipo": "Triángulo", "Vértices": [ {"x": "7.2", "y": "9.4"}, {"x": "8.1", "y": "6.9"}, {"x": "2.2", "y": "4.4"} ] }
```
- **Ventajas:**
 - Flexibilidad total en la estructura de datos.
 - Escalabilidad horizontal mediante sharding y replicación.
 - Índices sobre secciones de documentos para búsquedas rápidas.
 - Evitan JOINS al almacenar datos desnormalizados en un mismo documento.
- **Inconvenientes:**
 - No garantizan completamente ACID (suelen seguir el modelo BASE).
 - No permiten JOINS entre colecciones; la aplicación debe manejar las relaciones.
- **Ejemplos:** MongoDB, Apache CouchDB.

2.2. Bases de Datos Clave-Valor

- **Concepto:** Almacenan pares **clave-valor**, donde el valor es una secuencia de bytes arbitraria.
- **Funcionamiento:** Similar a un diccionario o array asociativo en programación.
- **Ejemplo de pares:**
 - Clave: `"imagen_1415.png"` → Valor: datos binarios de la imagen.
 - Clave: `"SELECT nombre FROM personas WHERE edad > 30"` → Valor: `["Luis", "Ana", "Sofía", "Juan"]`.
- **Ventajas:**
 - Máxima flexibilidad en los valores almacenados.
 - Alta escalabilidad y disponibilidad.
- **Inconvenientes:**
 - Sin capacidad de búsqueda dentro de los valores (solo por clave).
 - No soportan operaciones relacionales como JOINS.
- **Ejemplos:** Redis, Memcached, Berkeley DB.

2.3. Bases de Datos Columnares (o Orientadas a Columna)

- **Concepto:** Almacenan datos por **columnas** en lugar de por filas, optimizando el acceso a columnas específicas.
- **Ejemplo comparativo:**
- **Relacional (por filas):** [A3, Ana, Fernández, 32] , [B1, Luís, Cantador, 37]
- **Columnar (por columnas):** [A3, B1] , [Ana, Luís] , [Fernández, Cantador] , [32, 37]
- **Ventajas:**
 - Eficiencia en almacenamiento cuando hay muchos campos vacíos.
 - Lecturas rápidas al acceder solo a columnas necesarias.
 - Ideales para sistemas OLAP (procesamiento analítico).
- **Inconvenientes:**
 - Ineficientes para operaciones por fila (OLTP).
 - Difíciles para inserciones o actualizaciones frecuentes.
- **Ejemplos:** Google BigTable, Apache Cassandra, HBase.

2.4. Bases de Datos Orientadas a Grafo

- **Concepto:** Representan datos como **nodos** y **relaciones**, ambos con propiedades.
 - **Estructura:** Ideal para datos interconectados (ej.: redes sociales, motores de reglas).
 - **Ejemplo:** Un grafo donde los nodos son `Persona` y las relaciones `AMIGO_DE`.
 - **Ventajas:**
 - Excelentes para consultas complejas sobre relaciones.
 - Pueden soportar transacciones ACID.
 - **Inconvenientes:**
 - Difíciles de particionar en clústeres (el grafo debe caber en un nodo).
 - Ineficientes para datos sin relaciones complejas.
 - **Ejemplos:** Neo4j, AllegroGraph.
-

3. Primeros Pasos con MongoDB

Contexto práctico: Marta y María necesitan una base de datos para gestionar información heterogénea de niños con enfermedades raras. Una base de datos documental como MongoDB es ideal porque permite almacenar toda la información de cada niño en un único documento flexible.

3.1. Operaciones Básicas en MongoDB (CRUD)

- **use <db> :** Cambiar a una base de datos específica.
- **Inserción:** `db.coleccion.insert({documento})`
- **Consulta:** `db.coleccion.find()` para todos los documentos, o con filtros:
`db.coleccion.find({campo: valor})`
- **Proyección:** `db.coleccion.find({}, {campo: 1})` para seleccionar campos específicos.

3.2. Ejemplo de Uso

```
// Cambiar a la base de datos 'asociacion'
use asociacion

// Insertar un documento para un niño
db.ninos.insert({
  id: "N001",
  nombre: "Carlos",
  tratamientos: ["Terapia A", "Medicamento B"],
  pruebas: [
    { tipo: "Genética", resultado: "Positivo", fecha: "2023-10-01" }
  ]
})

// Buscar todos los niños
db.ninos.find()

// Buscar niños con un tratamiento específico
db.ninos.find({ tratamientos: "Terapia A" })
```

4. Conclusión y Aplicaciones Prácticas

- **Elección del tipo de NoSQL:** Depende del problema:
- **Documental:** Para datos heterogéneos y flexibles (ej.: perfiles de usuarios, catálogos de productos).
- **Clave-Valor:** Para cachés o sesiones de usuario.
- **Columnar:** Para análisis de grandes volúmenes de datos (ej.: data warehouses).
- **Grafo:** Para redes sociales, recomendaciones o sistemas de fraudes.
- **MongoDB en la práctica:** Es especialmente útil cuando los datos no tienen una estructura fija y se esperan cambios frecuentes en el esquema.

Recuerda: Aunque las bases de datos NoSQL ofrecen flexibilidad, es crucial planificar la estructura de los documentos o claves según las consultas más frecuentes para garantizar un rendimiento óptimo.

Análisis de una Consulta en MongoDB - Guía

Desglose de la Consulta Original

La consulta proporcionada es:

```
db.movies.find( { "awards.wins": { $gt: 100 } } );
```

Estructura de la Consulta:

- **db** : Referencia a la base de datos actual
- **movies** : Nombre de la colección dentro de la base de datos
- **find()** : Método para buscar documentos
- **{ "awards.wins": { \$gt: 100 } }** : Filtro de búsqueda

- `awards.wins` : Campo anidado (campo `wins` dentro del objeto `awards`)
- `$gt: 100` : Operador de comparación "greater than" (mayor que)

Opción Correcta Identificada

Opción 4: "Dentro de la colección movies de la base de datos db, encontrar documentos cuyo campo wins dentro del campo awards tenga valor mayor de 100."

Explicación Detallada:

1. **Base de datos y colección:** La consulta opera sobre la colección `movies` en la base de datos referenciada por `db`
2. **Campo anidado:** La notación `"awards.wins"` indica que se busca dentro de un campo llamado `awards` que contiene un subcampo llamado `wins`
3. **Operador de comparación:** `$gt: 100` significa "mayor que 100", no igual a 100

Análisis de las Opciones Incorrectas

Opción 1:

"Dentro de la base de datos movies, encontrar documentos cuyo campo wins tenga valor 100." - **Error:** Confunde base de datos con colección y no reconoce la estructura anidada del campo

Opción 2:

"Dentro de la colección movies de la base de datos db, encontrar documentos cuyo campo wins tenga valor 100." - **Error:** No reconoce que `wins` está anidado dentro de `awards` y usa igualdad (`=100`) en lugar de mayor que (`>100`)

Opción 3:

"Dentro de la colección movies de la base de datos db, encontrar documentos cuyo campo wins dentro del campo awards tenga valor 100." - **Error:** Identifica correctamente la estructura anidada pero usa igualdad (`=100`) en lugar de mayor que (`>100`)

Ejemplo Práctico

Supongamos un documento en la colección `movies` :

```
{
  "title": "The Best Movie",
  "awards": {
    "wins": 150,
    "nominations": 200
  }
}
```

Este documento **SÍ** sería devuelto por la consulta porque `awards.wins` (150) es mayor que 100.

Resumen de Aprendizaje

- **Notación de punto:** Se usa para acceder a campos anidados en MongoDB
- **Operadores de comparación:** `$gt` (mayor que), `$lt` (menor que), `$eq` (igual a)
- **Estructura de consultas:** `db.colección.método({filtro})`
- **Precisión en lectura:** Es crucial identificar correctamente tanto la estructura de campos como el operador de comparación usado

Conclusión: La consulta busca películas que hayan ganado más de 100 premios, donde la información de premios está almacenada en un campo anidado `awards.wins` .